

DAY 2 · 下午第二节 · STAGE 3: BUILD

方案已定，方向不变，专注执行

EVALUATE

THINK

PLAN

BUILD

REVIEW

VERIFY

BUILD 阶段不做方向性决策

入口：G2 通过 · 按 plan.md 写代码 + 写测试 · 发现方案有问题 → 回退 PLAN，不在 BUILD 里偷偷改方向

观察点：agent 是否参考 plan.md？是否遵循 TECH.md 约束？是否边写代码边写测试？

Agent 在 BUILD 阶段最爱犯的错

1. 偏离方案

「我觉得用异步更好」→ 方向性决策，
回退 PLAN

2. 过度工程

加了方案里没有的功能（「顺便加了缓存」）

3. 跳过测试

「代码写完了，测试等下补」→ 违反完成度 principle

4. 忽略错误处理

happy path 写了，异常路径没有

5. 硬编码

API key、超时时间等写死在代码里

6. 不参考 DDD

不读 PRODUCT.md 的业务规则就开始写

不要阻止偏差发生 —— 让它发生，记录它，这是系统学习的素材

纠正 → correction · corrections 积累 → 蒸馏原料 · 但灾难性错误（删库）必须拦住

G3 — 代码合格了吗?

G3 判定条件 (L1 自动化检查)

- ✓ 代码文件存在
- ✓ 测试存在且全部通过
- ✓ lint 通过 (如果配了)
- ✓ 无硬编码 secrets
- ✓ 实现覆盖 plan.md 中所有 AC 的映射

✓ **PASS**

进入 VERIFY 阶段

✗ FAIL 常见原因 → 处理

- 测试没写 / 没通过 → 原地修复
- lint 不过 → 原地修复
- AC 覆盖不全 → 补充实现
- 同一问题 3 次修不好 → 回退 PLAN (方案可能有问题)

G3 大部分条件可脚本化 —— 直接复用 `governance/gates/check-lint.sh` 等

EVALUATE

THINK

PLAN

BUILD

REVIEW

VERIFY

Stage 4: VERIFY — 「完成 = 我主动破坏它且失败了」

Verify Report: recognize 功能

AC 验证

[x] AC1: 5秒内返回 → 通过 (均 2.3s)
 [x] AC2: confidence 字段存在 → schema 过
 [] AC3: 低 confidence 标记待确认 → 未实现

破坏尝试

空图片 → 正确返回 400
 超大图片 → 内存溢出, 未处理
 非图片文件 → 正确返回 400
 Bedrock 超时 → 无超时处理, 永远挂起

判定

FAIL - 2 个 AC 未满足 + 2 个边界未处理

核心思维: 不是「检查它能跑」—— 是「尝试让它挂」

边界测试

- 空图片
- 超大图片 (10MB+)
- 非图片文件 (.txt 改 .jpg)
- 模糊 / 倾斜 / 手写
- 同时上传多张

异常路径测试

- Bedrock 超时
- Bedrock 返回异常格式
- 网络断开

这份报告 = G4 的判定依据

G4 不通过 → 回退 BUILD → 修 → 再 VERIFY

G4 判定

- ✓ verify artifact 存在
- ✓ 所有 AC 标记 pass
- ✓ 破坏尝试记录存在 (≥ 5 种)
- ✓ 无 fail 项

回退修复循环

BUILD → VERIFY → FAIL

↓ 回退

BUILD → VERIFY → FAIL

↓ ...

BUILD → VERIFY → PASS ✓

STATE.md

任务: recognize 函数开发

Profile: feature

当前阶段: DONE ✓

已通过 Gates:

G1 ✓ G2 ✓ G3 ✓ G4 ✓

循环次数: G3: 1次, G4: 2次

完成时间: 2024-01-15 16:45

第一次 FAIL ≠ 失败, 是系统在工作

通常 2-3 次循环可通过; 超过 3 次 → 问题可能在 PLAN 层面。循环不是浪费——它让 bug 在你这里被发现, 而不是在用户那里。

一天跑下来 —— 你多了什么

Engine 运行的产出

- spec/recognize/evaluate.md – 需求确认
- spec/recognize/plan.md – 技术方案 + ADR
- 代码实现 + 测试
- spec/recognize/verify.md – 验证报告

Knowledge 被填充

- TECH.md 新增 1-2 条 ADR
- corrections.log 大幅增长 (+10-20 条)
- 两天积累 → 可以做真正的蒸馏了

Engine 自身的反馈

哪个 gate 最容易 fail?

→ 那个环节 agent 偏差最大

有没有 gate 从来没 fail 过?

→ 可能不需要它

循环次数多少?

→ 越少说明 agent 越准

从空 Engine 到完整运行 + 真实产出

15-20 条 corrections —— 这是你明天蒸馏的原料。够了。

两天的 corrections —— 你能看到 pattern 吗？

快速蒸馏练习 (15 min)

1

打开 corrections.log

2

按类型分组：完成度 / 方案 / 引用 / 格式

3

问自己：principles 覆盖了哪些？有新 pattern 吗？有 rule 该退休吗？

4

如果发现需要 → 更新 governance

分组

典型条目

完成度类

跳过测试、验证太浅、说 done 太早

方案类

遗漏边界、不考虑异常

引用类

不读 DDD、不参考 plan

格式类

artifact 不完整

可能的蒸馏结果

- 新 Principle：关于「参考 DDD」（若反复不读文档）
- Rule 退休：G3 lint 已机械覆盖「代码格式」类 rule
- Principle 精炼：「完成度」措辞需更精确

DAY 2 · 完成

Engine + Knowledge 在联动了

Day 2 成就

Engine 设计完成 (stages + gates + profiles + state)

Engine 跑了一个完整 feature 流程

Knowledge 被 Engine 运行自然填充

corrections.log 15-20 条 (蒸馏原料充足)

做了一次 mini 蒸馏 (preview Day 3)

NEXT ► DAY 3

上午

验证工程 + Eval (怎么测 AgentOS 自身行为?)

上午

Loop Engineering (自动化运行 + 熔断)

下午

Capstone (整合、互评、Roadmap)

全景: Knowledge  有了真实内容 · Engine  设计+跑了一圈 · Eval ← 明天 | 今晚可选: 再读 corrections.log, 想想「跑一个月会变什么?」